PROGRAMMATION ASSEMBLEUR x86-64

Support de cours — Licence 3 (B3) — Suite (modules 2 à 10)

Adressage, arithmétique, flux de contrôle, pile, ABI, syscalls, chaînes et structures, protections mémoire, rétro-ingénierie, introduction SIMD

Dr Joffrey Catena

Ce support fait suite à la séance 1 (Module 1 — registres et formats de données). Il se lit en parallèle du workbook d'exercices (.md) : chaque module renvoie vers la phase d'exercices correspondante.

Sommaire

Séance 2 — 14h00–15h30 / pause / 15h45–17h15

- Module 2 — Modes d'adressage et modèle mémoire
- Module 3 — Instructions arithmétiques et logiques

Séance 3 — 8h00–9h30 / 9h45–11h15 / 11h30–13h00

- Module 4 — Flux de contrôle : comparaisons, sauts, boucles
- Module 5 — Pile, fonctions et convention d'appel System V AMD64
- Module 6 — Appels système et interfaçage avec le C

Pour aller plus loin — séances suivantes (à caler selon volume horaire restant)

- Module 7 — Chaînes, tableaux et structures de données
- Module 8 — Anatomie d'un cadre de pile, débordement de tampon et protections
- Module 9 — Lecture de code compilé : objdump, GDB et mini rétro-ingénierie
- Module 10 — Introduction à SIMD (SSE) et vectorisation

SÉANCE 2 — Créneau 1 : 14h00–15h30 | Pause : 15h30–15h45 | Créneau 2 : 15h45–17h15

Module 2 — Modes d'adressage et modèle mémoire

PHASE DE COURS 2 — 14h00–14h45

2.1 Les sections d'un binaire

Section	Contenu
.text	Code exécutable (instructions)
.data	Données initialisées (variables globales avec valeur)
.bss	Données non initialisées (réservation d'espace, resb/resq)
.rodata	Données en lecture seule (constantes)

Ces quatre sections se retrouvent dans presque tous les binaires, quel que soit le langage d'origine. .text est en lecture seule et exécutable, .data et .bss sont en lecture-écriture, .rodata est en lecture seule non exécutable. La différence entre .data et .bss est importante à souligner : .data occupe de la place dans le fichier binaire lui-même, alors que .bss n'occupe aucune place dans le fichier, seule la taille à réserver est indiquée, et c'est le système qui alloue la mémoire à zéro au lancement du programme.

2.2 Modes d'adressage

L'opérande mémoire générique en x86-64 suit la forme [base + index*échelle + déplacement]. Échelle appartient à {1, 2, 4, 8}.

Mode	Exemple	Signification
Immédiat	`mov rax, 5`	Charge la constante 5
Registre	`mov rax, rbx`	Copie rbx dans rax
Direct (mémoire)	`mov rax, [var]`	Charge la valeur à l'adresse var
Indirect par registre	`mov rax, [rbx]`	Charge la valeur à l'adresse contenue dans rbx
Base + déplacement	`mov rax, [rbx+8]`	Accès à un champ de structure
Base + index*échelle	`mov rax, [rbx+rcx*4]`	Accès à un tableau de int (4 octets)
Relatif à RIP	`mov rax, [rel var]`	Adressage PC-relatif (code indépendant de la position, PIE)

Le plus important à faire passer ici : les crochets [ ] signifient toujours « aller chercher en mémoire à cette adresse », alors que l'absence de crochets signifie « utiliser cette valeur directement ». En NASM, écrire mov rax, var (sans crochets) charge toujours l'adresse de var dans rax, jamais sa valeur : il n'y a pas d'ambiguïté contrairement à certains autres assembleurs. C'est mov rax, [var] qui charge la valeur stockée à cette adresse.

> **Attention :** depuis que les exécutables sont liés en PIE par défaut sur les distributions Linux récentes, une adresse globale ne peut plus être utilisée telle quelle dans le code : il faut systématiquement passer par l'adressage relatif à RIP, soit avec la directive default rel en tête de fichier, soit explicitement avec [rel var]. C'est ce que fait l'exemple ci-dessous.

Prenez l'exemple du tableau d'entiers en base+index*échelle : si rbx pointe le début d'un tableau de int (4 octets chacun) et rcx contient l'indice i, alors [rbx+rcx*4] est exactement l'équivalent assembleur de tableau[i] en C.

```
section .data
tab dd 10, 20, 30, 40      ; tableau de 4 entiers 32 bits
section .text
global _start
_start:
    lea rbx, [rel tab]     ; rbx = adresse de tab (compatible PIE)
    mov ecx, 2
    mov eax, [rbx+rcx*4]   ; eax = tab[2] = 30
    mov edi, eax           ; code de sortie = 30
```

```
    mov eax, 60
    syscall
```

Ce programme se termine avec un code de sortie égal à 30, vérifiable avec echo $? après exécution : c'est la valeur de tab[2] dans un tableau indexé à partir de 0.

2.3 LEA : calcul d'adresse sans accès mémoire

LEA (Load Effective Address) calcule une adresse selon les mêmes modes que ci-dessus mais sans déréférencer : c'est un outil de calcul arithmétique détourné, très utilisé pour multiplier ou additionner en une seule instruction.

```
lea rax, [rbx+rcx*2]   ; rax = rbx + 2*rcx (aucun accès mémoire !)
lea rax, [rax+rax*4]   ; rax = rax * 5
```

2.4 Déclaration de données

```
section .data
a   db 10                  ; byte initialisé
b   dw 1000                ; word
c   dd 100000               ; dword
d   dq 10000000000          ; qword
tab dd 1, 2, 3, 4           ; tableau de 4 dword
msg db "texte", 0
section .bss
buffer   resb 64            ; 64 octets non initialisés
compteur resq 1             ; 1 qword non initialisé
```

Les lettres après le nom de la variable (db, dw, dd, dq) indiquent la taille de chaque élément et acceptent une liste de valeurs séparées par des virgules, d'où tab dd 1, 2, 3, 4 qui déclare un tableau de 4 entiers 32 bits contigus en mémoire. À l'inverse, resb/resw/resd/resq (dans .bss) ne prennent pas de valeur initiale mais un nombre d'éléments à réserver : buffer resb 64 réserve 64 octets bruts, sans les initialiser, leur contenu est indéterminé tant qu'on ne les a pas écrits explicitement.

2.5 Alignement mémoire

L'ABI System V impose que RSP soit aligné sur 16 octets au moment d'un CALL (voir Module 5). Un mauvais alignement provoque des plantages aléatoires avec les instructions SSE/AVX qui exigent un alignement 16 octets (voir Module 10).

> **→ Traitez la Phase d'exercices 2 du workbook (adressage, LEA, tableaux).**

PAUSE — 15h30 à 15h45

Module 3 — Instructions arithmétiques et logiques

PHASE DE COURS 3 — 15h45–16h30

3.1 Transfert de données

Instruction	Effet
`mov dst, src`	dst = src
`movzx dst, src`	Copie avec extension par zéros (src plus petit que dst)
`movsx dst, src`	Copie avec extension de signe
xchg a, b	Échange a et b
`push src / pop dst`	Empile / dépile sur la pile

MOVZX et MOVSX servent au même problème, copier une petite valeur dans un registre plus grand, mais avec deux réponses différentes à la question de ce que l'on met dans les bits du haut. MOVZX les met à zéro : c'est ce qu'on veut pour une valeur non signée. MOVSX recopie le bit de signe : c'est ce qu'on veut pour préserver la valeur d'un entier signé négatif. Si AL contient 0xFF (soit -1 en signé sur 8 bits, ou 255 en non

signé), MOVZX EAX, AL donne EAX = 0x000000FF (255), alors que MOVSX EAX, AL donne EAX = 0xFFFFFFFF (-1). Le mauvais choix entre les deux est une source d'erreurs silencieuses très classique.

3.2 Arithmétique entière

```
add rax, rbx      ; rax += rbx
sub rax, rbx      ; rax -= rbx
inc rax           ; rax += 1
dec rax           ; rax -= 1
neg rax           ; rax = -rax
imul rax, rbx     ; rax = rax * rbx (signé)
imul rax, rbx, 5  ; rax = rbx * 5
; division : RDX:RAX / opérande -> quotient dans RAX, reste dans RDX
cqo               ; étend le signe de RAX dans RDX (RDX:RAX)
idiv rbx          ; division signée par rbx
```

> **Attention :** IDIV/DIV utilisent la paire RDX:RAX comme dividende 128 bits. Oublier d'initialiser ou d'étendre RDX (CQO/CDQ) avant une division est une erreur classique menant à une exception de division.

3.3 Logique binaire et décalages

```
and rax, rbx
or  rax, rbx
xor rax, rax      ; idiome standard pour mettre rax à 0 (plus rapide que mov rax,0)
not rax
shl rax, 1        ; décalage gauche = multiplication par 2
shr rax, 1        ; décalage droit logique (non signé)
sar rax, 1        ; décalage droit arithmétique (signé, préserve le signe)
```

Les deux décalent les bits vers la droite, mais SHR insère des zéros en tête alors que SAR recopie le bit de signe en tête. Utiliser SHR sur un nombre signé négatif transforme silencieusement un résultat négatif en un grand nombre positif : c'est un bug classique à surveiller.

3.4 Le registre RFLAGS

Indicateur	Nom	Positionné quand…
ZF	Zero Flag	Le résultat est nul
SF	Sign Flag	Le résultat est négatif (bit de poids fort = 1)
CF	Carry Flag	`Retenue/emprunt en arithmétique non signée`
OF	Overflow Flag	Débordement en arithmétique signée

CMP effectue une soustraction sans stocker le résultat, uniquement pour positionner les indicateurs, base des sauts conditionnels du Module 4.

> **→ Traitez la Phase d'exercices 3 du workbook (arithmétique, division, manipulation de bits).**

SÉANCE 3 — Créneau 1 : 8h00–9h30 | 9h45–11h15 | 11h30–13h00

Module 4 — Flux de contrôle : comparaisons, sauts, boucles

PHASE DE COURS 4 — 8h00–8h45

4.1 Sauts inconditionnels et conditionnels

jmp etiquette ; saut inconditionnel

Instruction	Condition (après CMP a, b)	Usage typique
`je / jz`	a == b (ZF=1)	égalité
`jne / jnz`	a != b	différence
`jg / jnle`	a > b (signé)	supérieur strict signé
`jge / jnl`	a >= b (signé)	supérieur ou égal signé
`jl / jnge`	a < b (signé)	inférieur strict signé
`jle / jng`	a <= b (signé)	inférieur ou égal signé
`ja / jnbe`	a > b (non signé)	supérieur strict non signé
`jb / jnae`	a < b (non signé)	inférieur strict non signé

> **Attention :** les comparaisons signées (jg/jl...) et non signées (ja/jb...) reposent sur des indicateurs différents (SF/OF vs CF). Se tromper de famille est une source classique de bugs silencieux.

4.2 Structures de contrôle classiques

Un point essentiel : l'assembleur n'a ni if, ni while, ni for. Toutes ces structures se traduisent, au niveau machine, par la même paire d'instructions : une comparaison (CMP) suivie d'un saut conditionnel.

if / else

```
    cmp rax, rbx
    jle else_branch
    ; bloc if
    jmp end_if
else_branch:
    ; bloc else
end_if:
```

Boucle while

```
while_cond:
    cmp rcx, 10
    jge end_while
    ; corps de boucle
    inc rcx
    jmp while_cond
end_while:
```

Boucle for avec un compteur décroissant

```
    xor eax, eax
    mov ecx, 10        ; compteur = nombre d'itérations
.boucle:
    add eax, ecx       ; corps de boucle : somme = somme + ecx
    dec ecx
    jnz .boucle        ; reboucle tant que ecx != 0
    ; eax contient 10+9+...+1 = 55
```

Ce code renvoie 55 comme code de sortie (10+9+8+...+1). Il existe aussi l'instruction LOOP, qui décrémente RCX et saute automatiquement si non nul, mais elle est plus lente que la paire dec/jnz sur les processeurs récents.

4.3 SETcc et CMOVcc

SETcc écrit 0 ou 1 dans un registre 8 bits selon une condition ; CMOVcc effectue un mouvement conditionnel sans branchement, utile pour éviter les pénalités de mauvaise prédiction de branchement.

```
cmp rax, rbx
setg cl        ; cl = 1 si rax > rbx, sinon 0
cmp rax, rbx
cmovg rdx, rax ; rdx = rax uniquement si rax > rbx
```

Pourquoi s'embêter avec CMOVcc alors qu'un simple JG ferait l'affaire ? Sur un processeur moderne, chaque saut conditionnel oblige le processeur à deviner à l'avance quelle branche va être prise pour continuer à exécuter des instructions en avance ; s'il se trompe, il doit tout annuler et recommencer, ce qui coûte une dizaine de cycles. CMOVcc évite complètement ce pari : les deux valeurs candidates sont déjà calculées, et l'instruction se contente de choisir laquelle garder.

> **→ Traitez la Phase d'exercices 4 du workbook (structures conditionnelles, boucles, recherche du maximum dans un tableau).**

Module 5 — Pile, fonctions et convention d'appel System V AMD64

PHASE DE COURS 5 — 9h45–10h30

5.1 La pile (stack)

La pile croît vers les adresses basses. RSP pointe le sommet. PUSH décrémente RSP de 8 puis écrit ; POP lit puis incrémente RSP de 8.

```
push rax   ; rsp -= 8 ; [rsp] = rax
pop rbx    ; rbx = [rsp] ; rsp += 8
```

En se représentant la mémoire avec les adresses hautes en haut et les adresses basses en bas, la pile descend visuellement vers le bas à chaque PUSH, et remonte à chaque POP. RSP contient une adresse, pas une valeur.

5.2 Convention d'appel System V AMD64 (Linux/macOS)

Élément	Registres / emplacement
Arguments entiers/pointeurs (dans l'ordre)	`RDI, RSI, RDX, RCX, R8, R9, puis pile`
Arguments flottants	`XMM0–XMM7`
Valeur de retour	`RAX (RAX:RDX si 128 bits)`
Préservés par l'appelé (callee-saved)	`RBX, RBP, RSP, R12–R15`
Non préservés (caller-saved)	`RAX, RCX, RDX, RSI, RDI, R8–R11`
Alignement RSP au moment du CALL	`RSP % 16 == 0 juste avant l'instruction CALL`

Deux lignes de ce tableau méritent un temps d'arrêt. D'abord caller-saved / callee-saved : un registre caller-saved peut être librement écrasé par la fonction appelée, donc c'est à l'appelant de le sauvegarder avant l'appel s'il en a encore besoin après ; un registre callee-saved doit au contraire être restitué intact par la fonction appelée avant qu'elle ne retourne. Ensuite, l'alignement de RSP sur 16 octets au moment du CALL : c'est une contrainte de l'ABI, pas du matériel, certaines fonctions C qui utilisent des instructions SSE/AVX en interne (comme printf) plantent ou produisent un résultat incorrect si cette règle n'est pas respectée.

5.3 Prologue et épilogue de fonction

```
ma_fonction:
    push rbp        ; sauvegarde l'ancien base pointer
    mov rbp, rsp    ; nouveau cadre de pile (stack frame)
    sub rsp, 32     ; espace pour variables locales (multiple de 16)
    ; ... corps de la fonction, variables locales via [rbp-8], [rbp-16]...
    mov rsp, rbp    ; libère les variables locales
    pop rbp         ; restaure l'ancien base pointer
    ret             ; retour (dépile l'adresse de retour posée par CALL)
```

Le prologue (push rbp / mov rbp, rsp / sub rsp, N) met en place le cadre de pile, et l'épilogue (mov rsp, rbp / pop rbp / ret) le démonte exactement dans l'ordre inverse. RBP sert de point de repère fixe pendant toute la durée de la fonction, ce qui permet d'accéder aux variables locales par un déplacement fixe sans avoir à recalculer leur position à chaque instruction. L'instruction leave (mov rsp, rbp suivi de pop rbp en une seule instruction) est un raccourci équivalent, utilisé dans l'exemple du Module 5.4 ci-dessous.

5.4 Exemple complet : fonction récursive (factorielle)

```
fact:                       ; int fact(int n) -- n en RDI
    push rbp
    mov rbp, rsp
    sub rsp, 16
    cmp rdi, 1
    jbe .base
    mov [rbp-8], rdi        ; sauvegarde n sur la pile
    dec rdi
    call fact               ; appel récursif fact(n-1)
    imul rax, [rbp-8]       ; rax = fact(n-1) * n
    jmp .fin
.base:
    mov eax, 1
.fin:
    leave                   ; équivaut à mov rsp,rbp + pop rbp
    ret
_start:
    mov edi, 5
    call fact               ; fact(5) = 120
    mov edi, eax
    mov eax, 60
    syscall
```

Ce code renvoie 120 (5!) comme code de sortie. La pile d'appels empile 5 cadres successifs avant que la récursion ne redescende.

5.5 Récursivité et débordement de pile

Chaque appel récursif empile un nouveau cadre de pile (adresse de retour + RBP sauvegardé + variables locales). Sans condition d'arrêt correcte, on obtient un débordement de pile (stack overflow), détectable via la page de garde (guard page) placée par le système. Ce mécanisme sera repris et creusé du point de vue sécurité au Module 8.

→ Traitez la Phase d'exercices 5 du workbook (fonctions, pile, récursivité, respect de l'ABI).

Module 6 — Appels système et interfaçage avec le C

PHASE DE COURS 6 — 11h30–12h15

6.1 Appels système Linux x86-64

Un syscall Linux se déclenche via l'instruction SYSCALL. Le numéro de service est placé dans RAX, les arguments dans RDI, RSI, RDX, R10, R8, R9 (R10 remplace RCX ici, car RCX est écrasé par SYSCALL).

N° (RAX)	Syscall	Arguments
0	read	`rdi=fd, rsi=buf, rdx=count`
1	write	`rdi=fd, rsi=buf, rdx=count`
2	open	`rdi=path, rsi=flags, rdx=mode`
3	close	`rdi=fd`
9	mmap	`rdi=addr, rsi=len, rdx=prot, r10=flags, r8=fd, r9=offset`
60	exit	`rdi=code_retour`

Cette table n'est qu'un extrait des quelque 350 syscalls Linux x86-64, mais ces six-là couvrent la majorité des besoins d'un programme simple.

6.2 Appeler une fonction C depuis l'assembleur

Grâce à l'ABI commune, on peut appeler printf, malloc, etc. directement depuis NASM en respectant la convention d'appel. Sur les systèmes récents, l'éditeur de liens produit par défaut des exécutables PIE (Position Independent Executable), qui interdisent les relocations absolues dans .text : il faut donc placer default rel en tête de fichier et appeler printf via la PLT.

```
default rel
extern printf
section .data
fmt db "Resultat : %d", 10, 0
section .text
global main
main:
    push rbp
    mov rbp, rsp
    lea rdi, [fmt]          ; 1er argument : format
    mov esi, 42             ; 2e argument : valeur
    xor eax, eax            ; 0 registre XMM utilisé (requis par printf variadique)
    call printf wrt ..plt
    xor eax, eax
    pop rbp
    ret

nasm -f elf64 prog.asm -o prog.o
gcc prog.o -o prog
./prog
```

> **Attention :** une ancienne forme (and rsp,-16 ; mov rdi,fmt ; call printf sans PLT, puis gcc prog.o -o prog) échoue à l'édition de liens sur une distribution récente avec l'erreur relocation R_X86_64_PC32 ... can not be used when making a PIE object : elle supposait un exécutable non-PIE. La version ci-dessus (default rel + wrt ..plt) fonctionne telle quelle avec un gcc moderne, sans option particulière. L'alternative revenant au comportement historique est de lier avec gcc -no-pie.

6.3 Appeler une fonction assembleur depuis le C

Symétriquement, une fonction assembleur respectant l'ABI (nom global, arguments en RDI/RSI..., retour en RAX) est directement appelable depuis un fichier C, en la déclarant extern côté C.

```
; carre.asm
section .text
global carre
carre:              ; int carre(int x)
    mov eax, edi
    imul eax, edi
    ret

// main.c
#include <stdio.h>
extern int carre(int x);
int main(void) {
    printf("%d\n", carre(6));  // affiche 36
    return 0;
}

nasm -f elf64 carre.asm -o carre.o
gcc main.c carre.o -o prog
./prog
```

Ce programme affiche 36. Si l'éditeur de liens affiche un avertissement missing .note.GNU-stack, il suffit d'ajouter en fin de fichier .asm la ligne section .note.GNU-stack noalloc noexec nowrite progbits : cet avertissement est bénin et sans conséquence.

6.4 Débogage avec GDB

- break *adresse ou break etiquette : point d'arrêt
- stepi / nexti : exécution pas à pas au niveau instruction

- info registers : affiche l'état des registres
- x/8gx $rsp : examine 8 quad-words au sommet de la pile en hexadécimal
- disassemble : affiche le désassemblage de la fonction courante

GDB est l'outil qui rend l'assembleur concret : au lieu de deviner ce qui se passe, on peut l'observer directement. Par exemple : poser un point d'arrêt sur une fonction (break nom_fonction), lancer le programme (run), puis avancer instruction par instruction (stepi) en vérifiant après chaque pas l'état des registres (info registers) ou le contenu de la pile (x/8gx $rsp).

→ Traitez la Phase d'exercices 6 du workbook (syscalls, interfaçage C ↔ ASM, projet de synthèse).

POUR ALLER PLUS LOIN — séances suivantes, à caler selon le volume horaire restant

Les modules 7 à 10 prolongent le programme au-delà du support initial. Ils reprennent systématiquement des notions déjà vues (adressage, pile, ABI) pour les appliquer à des problématiques plus proches du cœur de métier : lecture de chaînes et de structures, sécurité mémoire, rétro-ingénierie, calcul vectoriel. Chaque module est calibré pour une séance de 3 heures (deux créneaux de 1h30), ajustable selon le volume réellement disponible.

Module 7 — Chaînes, tableaux et structures de données

PHASE DE COURS 7

7.1 Chaînes de caractères en C : rappel du modèle

En assembleur comme en C, une chaîne est une simple suite d'octets en mémoire terminée par un octet nul (0x00). Il n'existe pas de type chaîne au niveau machine : tout est affaire de convention et de parcours mémoire.

7.2 Instructions de chaînes : MOVS, SCAS, CMPS, REP

Le jeu d'instructions x86 propose des instructions dédiées au parcours de blocs mémoire, pilotées par RSI (source), RDI (destination) et RCX (compteur), combinées au préfixe REP qui répète l'instruction tant que RCX n'est pas nul.

Instruction	Effet	Avec REP
lodsb/w/d/q	charge [rsi] dans AL/AX/EAX/RAX, puis rsi++	rare
stosb/w/d/q	écrit AL/AX/EAX/RAX dans [rdi], puis rdi++	initialise un buffer
`movsb/w/d/q`	copie [rsi] vers [rdi], rsi++ et rdi++	copie un bloc mémoire
scasb/w/d/q	compare AL/AX/EAX/RAX à [rdi], puis rdi++	`repne scasb : recherche un octet`
`cmpsb/w/d/q`	compare [rsi] à [rdi], rsi++ et rdi++	`repe cmpsb : compare deux blocs`

Le sens du parcours (adresses croissantes ou décroissantes) dépend du Direction Flag (DF), positionné par CLD (DF=0, sens croissant, le cas normal) ou STD (DF=1). Toujours placer CLD en tête de fonction utilisant ces instructions, car l'ABI ne garantit pas l'état de DF à l'entrée.

7.3 Implémenter strlen soi-même

```
; rdi = adresse de la chaîne ; retour : rax = longueur
mon_strlen:
    xor eax, eax
    mov rcx, -1       ; compteur maximal (décrémenté jusqu'à trouver 0)
    mov rdx, rdi      ; conserve l'adresse de départ
    cld
    repne scasb       ; cherche l'octet 0 à partir de [rdi], rcx--, rdi++ à chaque pas
    mov rax, rdi
    sub rax, rdx
    dec rax           ; ne pas compter l'octet nul
    ret
```

Sur « bonjour b3 » (10 caractères), cette fonction renvoie bien 10. REPNE SCASB est en réalité une boucle cmp + jump déguisée (voir Module 3), exécutée en microcode.

7.4 Transformation d'une chaîne : mise en majuscules

```
; rsi = source (terminée par 0), rdi = destination
.boucle:
    lodsb              ; al = [rsi], rsi++
    test al, al
    jz .fin
    cmp al, 'a'
    jb .ecrire
```

```
    cmp al, 'z'
    ja .ecrire
    sub al, 32          ; 'a'-'A' = 32 : bascule minuscule -> majuscule
.ecrire:
    stosb               ; [rdi] = al, rdi++
    jmp .boucle
.fin:
```

« bonjour b3 » devient « BONJOUR B3 ». Cette astuce arithmétique (soustraire 32) ne fonctionne que parce que les blocs majuscules et minuscules sont alignés et contigus dans la table ASCII.

7.5 Lire les arguments de la ligne de commande (argv)

Sous Linux, au tout début de _start, la pile contient dans l'ordre : argc, puis argv[0], argv[1], ..., puis un pointeur nul, puis envp. Aucun registre n'est préparé automatiquement : il faut lire directement [rsp].

```
_start:
    mov rdi, [rsp]        ; argc
    cmp rdi, 2
    jl .quitter
    mov rsi, [rsp+16]     ; argv[1]  (argv[0] est à [rsp+8])
    ; ... utiliser rsi comme adresse de chaîne
.quitter:
    mov eax, 60
    xor edi, edi
    syscall
```

Avec ./prog salut, le programme retrouve correctement argv[1]. C'est exactement ce que le runtime C fait avant d'appeler main(argc, argv) (voir Module 6).

7.6 Structures de données : l'équivalent assembleur d'un struct C

NASM fournit les directives struc/endstruc pour nommer les décalages d'une structure, exactement comme un struct en C.

```
struc personne
    .id     resd 1
    .age    resb 1
    .pad    resb 3      ; octets de bourrage pour l'alignement
    .solde  resq 1
endstruc

section .data
p1: istruc personne
    at personne.id,    dd 7
    at personne.age,   db 33
    at personne.solde, dq 1500
iend

section .text
    lea rbx, [rel p1]
    mov eax, [rbx + personne.id]          ; eax = 7
    movzx ecx, byte [rbx + personne.age]  ; ecx = 33
    mov rdx, [rbx + personne.solde]       ; rdx = 1500
```

Avec id=7, age=33 et personne_size=16, ce code renvoie 56 (16+7+33), confirmant les décalages .id, .age, .solde et la taille totale personne_size. Le compilateur aligne .solde (un qword) sur une frontière de 8 octets, d'où les 3 octets de bourrage après .age (le champ .pad).

> **→ Traitez la Phase d'exercices 7 du workbook (strlen, inversion de chaîne, recherche de caractère, structure personne).**

Module 8 — Anatomie d'un cadre de pile, débordement de tampon et protections

PHASE DE COURS 8

8.1 Reprendre le cadre de pile du Module 5, vu sous l'angle sécurité

Dans le prologue push rbp / mov rbp, rsp / sub rsp, N, l'espace réservé par sub rsp contient les variables locales, dont d'éventuels buffers. Juste au-dessus de ce cadre se trouvent l'ancien RBP sauvegardé, puis l'adresse de retour déposée par CALL : c'est précisément cette zone qu'un débordement de tampon mal maîtrisé peut écraser.

8.2 Principe d'un débordement de tampon (stack buffer overflow)

Si une fonction copie des données dans un buffer local sans vérifier leur taille (par exemple avec strcpy en C, ou une boucle d'écriture sans borne en assembleur), une entrée trop longue peut déborder au-delà du buffer et écraser l'ancien RBP puis l'adresse de retour. Au moment du ret, le processeur dépile cette adresse corrompue et y saute : c'est le mécanisme à la base d'une classe entière de vulnérabilités historiques.

> **Attention :** ce module reste volontairement au niveau explicatif : l'objectif est de comprendre pourquoi les protections ci-dessous existent, pas de construire un exploit fonctionnel.

8.3 Les protections mises en place par les systèmes modernes

Protection	Principe	Ce qu'elle empêche
Stack canary	Valeur aléatoire déposée après le prologue, vérifiée avant le ret	Détecte un débordement avant que le ret corrompu ne s'exécute
NX / DEP (No-eXecute)	Les pages de pile et de tas sont marquées non exécutables	Empêche d'exécuter du code injecté directement sur la pile
ASLR	L'adresse de base de la pile, du tas et des bibliothèques est aléatoire à chaque exécution	Empêche de viser une adresse de retour fixe connue à l'avance
RELRO / PIE	Sections réordonnées en lecture seule, exécutable rendu position-indépendant	Réduit les cibles exploitables (GOT en lecture seule notamment)

Ces quatre protections se combinent : un stack canary détecte le débordement lui-même, NX empêche d'exécuter du code injecté même si l'adresse de retour est détournée, et ASLR complique le choix d'une adresse de redirection valide. C'est exactement ce qui explique l'évolution des exploits réels, du simple shellcode sur la pile des années 1990-2000 vers les techniques de contournement bien plus sophistiquées (ROP, etc.) apparues en réponse à ces protections, qui restent hors du périmètre de ce cours.

8.4 Observer un canary avec GDB

Un binaire C compilé normalement (gcc sans -fno-stack-protector) place un canary visible dans son désassemblage. En compilant un petit programme C avec un buffer local puis en désassemblant main avec objdump -d ou gdb, on repère le mov rax, fs:0x28 (lecture du canary) en début de fonction et sa vérification juste avant le ret.

> **→ Traitez la Phase d'exercices 8 du workbook (identifier la taille d'un cadre de pile vulnérable, associer chaque protection à ce qu'elle empêche, lecture d'un désassemblage annoté).**

Module 9 — Lecture de code compilé : objdump, GDB et mini rétro-ingénierie

PHASE DE COURS 9

9.1 De la source au binaire : les étapes

gcc -S prog.c produit le fichier assembleur généré par le compilateur (syntaxe AT&T par défaut, ou Intel avec -masm=intel), avant assemblage. C'est le moyen le plus direct de voir comment le compilateur traduit if, while et les appels de fonction dans les instructions vues aux Modules 4 et 5.

9.2 objdump : désassembler un binaire existant

```
objdump -d prog                # désassemblage de toutes les sections exécutables
objdump -d prog -M intel       # en syntaxe Intel (celle de ce cours)
objdump --disassemble=fonction prog   # une seule fonction

0000000000401000 <fact>:
  401000: 55                    push   rbp
  401001: 48 89 e5              mov    rbp,rsp
  401004: 48 83 ec 10           sub    rsp,0x10
  401008: 48 83 ff 01           cmp    rdi,0x1
  40100c: 76 13                 jbe    401021 <fact.base>
  40100e: 48 89 7d f8           mov    QWORD PTR [rbp-0x8],rdi
  401012: 48 ff cf              dec    rdi
  401015: e8 e6 ff ff ff        call   401000 <fact>
```

Cette capture est issue du désassemblage réel de la fonction fact du Module 5. Essayez d'y reconnaître, sans aide, le prologue (push rbp / mov rbp,rsp / sub rsp,0x10) puis le test d'arrêt de la récursion (cmp rdi,0x1 / jbe) : l'objectif est de retrouver la structure logique du Module 5 dans un désassemblage brut, sans le code source sous les yeux.

9.3 Démarche de rétro-ingénierie d'un petit binaire

- 1. file prog : identifier le format (ELF 64 bits, PIE ou non, lié dynamiquement ou statiquement)
- 2. objdump -d prog -M intel : repérer les fonctions et leur nom (si le binaire n'est pas strippé)
- 3. Identifier les prologues/épilogues pour délimiter chaque fonction
- 4. Repérer les CALL vers des fonctions connues (printf, strcmp...) pour déduire le rôle du code environnant
- 5. Charger dans GDB, poser des points d'arrêt aux frontières de fonctions repérées, et confirmer l'hypothèse pas à pas (stepi, info registers)

9.4 Exercice guidé : retrouver la logique d'une fonction mystère

Vous recevrez un binaire compilé sans son code source (une fonction simple : comparaison, boucle, ou petit calcul), à désassembler puis à annoter ligne par ligne, avant de proposer une réécriture en C équivalente.

> **→ Traitez la Phase d'exercices 9 du workbook (lecture de désassemblages annotés, binaire mystère à rétro-documenter).**

Module 10 — Introduction à SIMD (SSE) et vectorisation

PHASE DE COURS 10

10.1 Pourquoi le calcul vectoriel

Les instructions vues jusqu'ici (Modules 1 à 9) traitent une donnée scalaire à la fois. Les registres XMM (128 bits), déjà croisés au Module 1 comme usage conventionnel de la convention d'appel pour les arguments flottants, permettent d'appliquer la même opération à plusieurs données simultanément : c'est le principe SIMD (Single Instruction, Multiple Data).

10.2 Registres XMM et alignement

Un registre XMM de 128 bits peut contenir 4 flottants simple précision (32 bits chacun) ou 2 flottants double précision (64 bits chacun). Les instructions comme MOVAPS exigent un alignement mémoire sur 16 octets (d'où la directive align 16 vue au Module 2), faute de quoi elles déclenchent une exception.

10.3 Exemple : addition de deux vecteurs de 4 flottants

```
section .data
align 16
a dd 1.0, 2.0, 3.0, 4.0
```

```
align 16
b dd 10.0, 20.0, 30.0, 40.0
align 16
r dd 0.0, 0.0, 0.0, 0.0
section .text
global _start
_start:
    movaps xmm0, [a]   ; charge les 4 flottants de a dans xmm0
    movaps xmm1, [b]   ; charge les 4 flottants de b dans xmm1
    addps  xmm0, xmm1  ; addition des 4 paires en une seule instruction
    movaps [r], xmm0   ; r = {11.0, 22.0, 33.0, 44.0}
    mov eax, 60
    xor edi, edi
    syscall
```

À la différence du Module 3, ADDPS additionne ici 4 paires de flottants en une seule instruction, là où 4 ADD scalaires successifs seraient nécessaires sans SIMD. C'est le principe que les compilateurs exploitent automatiquement via l'auto-vectorisation (gcc -O3).

10.4 Ouverture : où cela mène ensuite

Ce module n'est qu'une introduction. Les extensions AVX/AVX2 (registres YMM de 256 bits) et AVX-512 prolongent le même principe avec des registres plus larges ; elles sont omniprésentes dans le calcul scientifique, le traitement d'image et le chiffrement optimisé, ce qui en fait un prolongement naturel pour les étudiants intéressés par la cryptographie ou la performance bas niveau.

> **→ Traitez la Phase d'exercices 10 du workbook (addition et multiplication vectorielles, comparaison de performance scalaire vs SIMD).**